

gRPC

gRPC — JUN 2024 — Servis za upravljanje listom poruka

Zadatak: U .NET-u, koristeći gRPC, kreirati servis za upravljanje listom poruka. Servis treba da omogući klijentima da dodaju i brišu poruke, i prikažu sve poruke. Definirati proto fajl (message.proto) koji daje specifikaciju servisa i poruka. Servis treba da podržava sledeće operacije: SendMessage — dodaje novu poruku na listu; DeleteMessage — briše poruku sa zadatim ID-jem; ListMessages — dobija tok podataka koji sadrži sve poruke. Implementirati gRPC server definisan u message.proto fajlu.

message.proto

```
syntax = "proto3";
package Messages;

service MessageService {
  rpc SendMessage(Text) returns (Response);
  rpc DeleteMessage(TextId) returns (Response);
  rpc ListMessages(EmptyRequest) returns (stream TextList);
}

message Text {
  string msg = 1;
}

message TextId {
  int32 id = 1;
}

message EmptyRequest {}

message Response {
  int32 textId = 1;
  string text = 2;
  string resMsg = 3;
  bool success = 4;
}

message TextList {
  int32 textId = 1;
  string text = 2;
}
```

MessageServiceImpl.cs

```
using Grpc.Core;
using Messages;

public class MessageServiceImpl : MessageService.MessageServiceBase {
  private static int textId = 0;
  private static List<TextList> lists = new List<TextList>();

  public override Task<Response> SendMessage(Text request, ServerCallContext ctx) {
    int id = textId++;

    lists.Add(new TextList{
      TextId = id,
      Text = request.Msg
    });

    return Task.FromResult(new Response{
      TextId = id,
```

```

        Text = request.Msg,
        ResMsg = "Uspesno kreirana poruka",
        Success = true
    });
}

public override Task<Response> DeleteMessage(TextId request, ServerCallContext ctx) {
    var context = lists.Find(m => m.TextId == request.Id);
    if(context != null) {
        lists.Remove(context);

        return Task.FromResult(new Response{
            TextId = context.TextId,
            Text = context.Text,
            ResMsg = "Uspesno obrisana poruka",
            Success = true
        });
    } else {
        return Task.FromResult(new Response{
            TextId = -1,
            Text = "",
            ResMsg = "Poruka ne postoji",
            Success = false
        });
    }
}

public override async Task ListMessages(EmptyRequest request, IServerStreamWriter<Text
    List> response, ServerCallContext ctx) {
    foreach (var t in lists) {
        await response.WriteAsync(t);
    }
}
}

```

gRPC — OKT2 2025 — Tok podataka — da li je b kvadrat broja a

Zadatak: U .NET-u, koristeći gRPC, napisati servis koji prihvata tok podataka, gde se svaki podatak sastoji od dva prirodna broja a i b. Za svaku primljenu poruku, procedura ispituje da li je broj b kvadrat broja a, i vraća nazad klijentu tok poruka sa sadržajem „Da“, ukoliko broj b jeste kvadrat broja a, ili „Ne“, ukoliko broj b nije kvadrat broja a. Napisati i proto fajl (okt2.proto) koji daje specifikaciju servisa i poruka.

okt2.proto

```
syntax = "proto3";
package Okt2;

service Okt2Service {
  rpc isQuadro(stream Input) returns (stream Response);
}

message Input {
  int32 a = 1;
  int32 b = 2;
}

message Response {
  string text = 1;
}
```

Okt2ServiceImpl.cs

```
using Grpc.Core;
using Okt2;

public class OktServiceImpl : Okt2Service.Okt2ServiceBase {

    public override async Task isQuadro(IAsyncStreamReader<Input> request, IServerStreamWriter<Response> response, ServerCallContext ctx) {
        await foreach (var r in request.ReadAllAsync()) {
            if (r.A * r.A == r.B) {
                await response.WriteAsync(new Response { Text = "Da" });
            } else {
                await response.WriteAsync(new Response { Text = "Ne" });
            }
        }
    }
}
```

gRPC — JUN 2026 — Kalkulator — suma N brojeva i množenje 2 broja

Zadatak: U .NET-u, koristeći gRPC, napisati servis koji simulira rad kalkulatora koji podržava operacije sabiranja N celih brojeva i množenja 2 cela broja. Operacija sabiranja prihvata redom sve cele brojeve, računa njihovu sumu i vraća je nazad kao odgovor klijentu, pri čemu je klijentu potrebno vratiti i broj celih brojeva koji je učestvovao u formiranju te sume. Operacija množenja prihvata dva cela broja i vraća odgovor koji treba sadržati polja operand1, operand2 i rezultat. Definirati i proto fajl (jun.proto) koji daje specifikaciju servisa i poruka.

jun.proto

```
syntax = "proto3";
package Calculator;

service CalculatorService {
    rpc sum(stream SumNumReq) returns (SumNumRes);
    rpc multiply(MulNumReq) returns (MulNumRes);
}

message SumNumReq {
    int32 broj = 1;
}

message SumNumRes {
    int32 suma = 1;
    int32 vrednost = 2;
}

message MulNumReq {
    int32 operand1 = 1;
    int32 operand2 = 2;
}

message MulNumRes {
    int32 operand1 = 1;
    int32 operand2 = 2;
    int32 rezultat = 3;
}
```

CalculatorServiceImpl.cs

```
using Grpc.Core;
using Calculator;

public class CalculatorServiceImpl : CalculatorService.CalculatorServiceBase {

    public override async Task<SumNumRes> sum(IAsyncStreamReader<SumNumReq> request, ServerCallContext ctx) {
        int suma = 0;
        int vrednost = 0;

        await foreach (var r in request.ReadAllAsync()) {
            suma += r.Broj;
            vrednost++;
        }

        return new SumNumRes { Suma = suma, Vrednost = vrednost };
    }

    public override Task<MulNumRes> multiply(MulNumReq request, ServerCallContext ctx) {
        return Task.FromResult(new MulNumRes {
            Operand1 = request.Operand1,
            Operand2 = request.Operand2,
            Rezultat = request.Operand1 * request.Operand2
        });
    }
}
```

```
        Resultat = request.Operand1 * request.Operand2
    });
}
```

Java RMI

Java RMI — JUN 2024 — MQTT broker (isti zadatak: OKT2 2025 i JUL 2026)

Zadatak: Korišćenjem Java RMI tehnologije napisati simulaciju MQTT brokera. MQTT broker je server koji razmenjuje poruke između klijenata korišćenjem topika. Topik predstavlja string na osnovu koga se poruke (definisane naslovom i sadržajem) filtriraju i dostavljaju određenim klijentima. Neophodno je implementirati metodu subscribe kako bi se klijent pretplatio na poruke pristigle na određeni topik, kao i metodu publish koja omogućava korisniku da pošalje poruku na željeni topik. Klijent ne kreira topik eksplicitno, već se topik, ukoliko ne postoji, kreira implicitno pozivom metode createTopic. Napisati serversku aplikaciju za hostovanje MQTT brokera i registrovati ga u RMI registar, kao i klijentsku aplikaciju koja će minimalno demonstrirati funkcionisanje istog.

Broker.java

```
public interface Broker extends Remote {
    void subscribe(String topic, ClientCallback cbk) throws RemoteException;
    void publish(String topic, Message msg) throws RemoteException;
}
```

ClientCallback.java

```
public interface ClientCallback extends Remote {
    void notify(String topic, Message msg) throws RemoteException;
}
```

Message.java

```
public class Message implements Serializable {
    private String title;
    private String content;

    public Message(String title, String content) {
        this.title = title;
        this.content = content;
    }

    public String toString() {
        return "Poruka [" + this.title + "] " + this.content;
    }
}
```

BrokerImpl.java

```
public class BrokerImpl extends UnicastRemoteObject implements Broker {
    private Map<String, Set<ClientCallback>> topics = new HashMap<>();

    public BrokerImpl() throws RemoteException {
        super();
    }

    public void subscribe(String topic, ClientCallback cbk) throws RemoteException {
        this.createTopic(topic);

        this.topics.get(topic).add(cbk);
    }

    public void publish(String topic, Message msg) throws RemoteException {
        this.createTopic(topic);
        for(ClientCallback c : this.topics.get(topic)) {

```

```

        c.notify(topic, msg);
    }
}

private void createTopic(String topic) {
    this.topics.putIfAbsent(topic, new HashSet<>());
}
}

```

Server.java

```

public class Server {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099);

            Broker broker = new BrokerImpl();

            Naming.rebind("rmi://localhost:1099/BrokerService", broker);

            System.out.println("Server started");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Client.java

```

public class Client {
    public static void main(String[] args) {
        try {
            Broker broker = (Broker) Naming.lookup("rmi://localhost:1099/BrokerService");

            ClientCallback cbk = new ClientCallbackImpl();

            broker.subscribe("cs16esport", cbk);

            broker.publish("cs16esport", new Message("cs16esport", "najjaca igrica svih vremena"));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static class ClientCallbackImpl extends UnicastRemoteObject implements ClientCallback {
        public ClientCallbackImpl() throws RemoteException {
            super();
        }

        public void notify(String topic, Message msg) throws RemoteException {
            System.out.println("Nova poruka u " + topic + " : " + msg.toString());
        }
    }
}

```

Java RMI — JUN 2026 — Online enciklopedija

Zadatak: Korišćenjem Java RMI tehnologije implementirati simulaciju online enciklopedije. Klasa Encyclopedia treba da čuva informacije o svim člancima koji postoje u sistemu. Potrebno je implementirati metodu `getArticlesTitles()` koja će korisniku vratiti String literal sa naslovima svih članaka u sistemu, kao i metodu `getArticleByTitle(String title)` koja korisniku treba vratiti članak sa prosleđenim naslovom. U konstruktoru klase Encyclopedia kreirati dva članka i smestiti ih u odgovarajuću strukturu podataka. Članak je definisan klasom Article (private String title, private String content, private LocalDateTime lastUpdated) koja treba omogućiti korisniku da preuzme informacije o članku metodom `String info()`, kao i da promeni sadržaj članka metodom `updateContent(String newContent)`. Napisati serversku aplikaciju za hostovanje enciklopedije i registrovati je u RMI registar na portu 5000, kao i klijentsku aplikaciju koja će minimalno demonstrirati funkcionisanje iste.

Encyclopedia.java

```
public interface Encyclopedia extends Remote {
    String getArticlesTitles() throws RemoteException;
    Article getArticleByTitle(String title) throws RemoteException;
}
```

Article.java

```
public interface Article extends Remote {
    String info() throws RemoteException;
    void updateContent(String newContent) throws RemoteException;
}
```

ArticleImpl.java

```
public class ArticleImpl extends UnicastRemoteObject implements Article {
    private String title;
    private String content;
    private LocalDateTime lastUpdated;

    public ArticleImpl(String title, String content) throws RemoteException {
        super();
        this.title = title;
        this.content = content;
        this.lastUpdated = LocalDateTime.now();
    }

    public String info() throws RemoteException {
        return "Naslov : " + this.title + " Content : " + this.content + " Updated : " + this.lastUpdated.toString();
    }

    public void updateContent(String newContent) throws RemoteException {
        this.content = newContent;
        this.lastUpdated = LocalDateTime.now();
    }
}
```

EncyclopediaImpl.java

```
public class EncyclopediaImpl extends UnicastRemoteObject implements Encyclopedia {
    private Map<String, Article> articles = new HashMap<String, Article>();

    public EncyclopediaImpl() throws RemoteException {
        super();

        Article a1 = new ArticleImpl("Java RMI", "Java RMI Content");
```



```

        Article a2 = new ArticleImpl("NestJS", "JavaScript is the best");

        this.articles.put("Java RMI", a1);
        this.articles.put("NestJS", a2);
    }

    public String getArticlesTitles() throws RemoteException {
        String titles = "";

        for(String t : articles.keySet()) {
            titles += t + " , ";
        }

        return titles;
    }

    public Article getArticleByTitle(String title) throws RemoteException {
        return articles.get(title);
    }
}

```

Server.java

```

public class Server {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(5000);

            Encyclopedia enc = new EncyclopediaImpl();

            Naming.rebind("rmi://localhost:5000/EncyclopediaService", enc);

            System.out.println("Server started");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Client.java

```

public class Client {
    public static void main(String[] args) {
        try {
            Encyclopedia enc = (Encyclopedia) Naming.lookup("rmi://localhost:5000/EncyclopediaService");

            System.out.println("Naslovi : " + enc.getArticlesTitles());

            System.out.println("Article 1 : " + enc.getArticleByTitle("Java RMI").info());

            Article a1 = enc.getArticleByTitle("Java RMI");
            a1.updateContent("New content");
            System.out.println("New contenttttzz : " + a1.info());

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Java RMI — VEŽBA — Online aukcija (sa callback obaveštavanjem)

Zadatak: Korišćenjem Java RMI tehnologije implementirati simulaciju sistema za onlajn aukciju. Klasa AuctionHouse treba da čuva informacije o svim predmetima koji se trenutno prodaju, kao i o trenutno najvišoj ponudi za svaki predmet. Ponuda je definisana klasom Bid (private String bidder, private double amount). Potrebno je implementirati metodu registerBidder kojom se klijent prijavljuje za praćenje određenog predmeta, kao i metodu placeBid kojom klijent daje ponudu za zadati predmet. Ponuda se prihvata samo ukoliko je viša od trenutno najviše ponude za taj predmet — metoda vraća true ako je ponuda prihvaćena, odnosno false u suprotnom. Kada je ponuda prihvaćena, svi klijenti koji prate taj predmet moraju biti obavešteni o novoj najvišoj ponudi. Predmet se ne kreira eksplicitno, već se, ukoliko ne postoji, kreira implicitno pozivom metode createItem. Napisati serversku aplikaciju za hostovanje aukcijske kuće i registrovati je u RMI registar na portu 4000, kao i klijentsku aplikaciju koja će minimalno demonstrirati funkcionisanje istog.

AuctionHouse.java

```
public interface AuctionHouse extends Remote {
    void registerBidder(String subject, ClientCallback cbk) throws RemoteException;
    boolean placeBid(String subject, Bid bid) throws RemoteException;
}
```

ClientCallback.java

```
public interface ClientCallback extends Remote {
    void notify(String subject, Bid bid) throws RemoteException;
}
```

Bid.java

```
public class Bid implements Serializable {
    private String bidder;
    private double amount;

    public Bid(String bidder, double amount) {
        this.bidder = bidder;
        this.amount = amount;
    }

    public double getAmount() {
        return this.amount;
    }

    public String toString() {
        return "Bidder " + this.bidder + " bid " + this.amount;
    }
}
```

AuctionHouseImpl.java

```
public class AuctionHouseImpl extends UnicastRemoteObject implements AuctionHouse {
    private Map<String, Double> bids = new HashMap<>();
    private Map<String, Set<ClientCallback>> subscribers = new HashMap<>();

    public AuctionHouseImpl() throws RemoteException {
        super();
    }

    public void registerBidder(String subject, ClientCallback cbk) throws RemoteException {
        {
            this.createItem(subject);
            this.subscribers.get(subject).add(cbk);
        }
    }
}
```

```

    }

    public boolean placeBid(String subject, Bid bid) throws RemoteException {
        this.createItem(subject);

        if(bid.getAmount() > this.bids.get(subject)) {
            this.bids.put(subject, bid.getAmount());

            for (ClientCallback c : this.subscribers.get(subject)) {
                c.notify(subject, bid);
            }

            return true;
        }

        return false;
    }

    private void createItem(String subject) {
        this.bids.putIfAbsent(subject, 0.0);
        this.subscribers.putIfAbsent(subject, new HashSet<>());
    }
}

```

Server.java

```

public class Server {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(4000);

            AuctionHouse ah = new AuctionHouseImpl();

            Naming.rebind("rmi://localhost:4000/AuctionHouseService", ah);

            System.out.println("Server started");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Client.java

```

public class Client {
    public static void main(String[] args) {
        try {
            AuctionHouse ah = (AuctionHouse) Naming.lookup("rmi://localhost:4000/AuctionHouseService");

            ClientCallback cbk = new ClientCallbackImpl();

            String subject = "cs16";

            ah.registerBidder(subject, cbk);

            ah.placeBid(subject, new Bid("bot", 100.0));

            ah.placeBid(subject, new Bid("cofyye", 199291929.22));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static class ClientCallbackImpl extends UnicastRemoteObject implements ClientCallback {
        public ClientCallbackImpl() throws RemoteException {

```

```
        super();
    }

    public void notify(String subject, Bid bid) throws RemoteException {
        System.out.println("Subject " + subject + "have a new bidder, it is :" + bid.toString());
    }
}
```

MPI

PODSETNIK (smernice): pojedinačni pokazivači = MPI_File_seek + MPI_File_read/MPI_File_write (uvek ide seek). Eksplicitni pomeraj = MPI_File_read_at / MPI_File_write_at (NE ide seek). Offset unazad: $(size - 1 - rank) \times N \times \text{sizeof}(\text{tip})$. Offset napred: $rank \times N \times \text{sizeof}(\text{tip})$. bufsize = FILESIZE / nprocs. 1kB = 1024, 1MB = 1024×1024, 10MB = 10×1024×1024.

MPI — JUN 2024 — Paralelni upis i čitanje binarne datoteke (prvi deo: a + b)

Zadatak: Napisati MPI program koji vrši paralelni upis i čitanje binarne datoteke, prema sledećim zahtevima: a) Svaki proces upisuje po 105 proizvoljnih celih brojeva u datoteku file1.dat. Upis se vrši upotrebom pojedinačnih pokazivača, dok redosled podataka u fajlu ide od podataka poslednjeg do podataka prvog procesa. b) Ponovo otvoriti datoteku. Svaki proces vrši čitanje upravo upisanih podataka upotrebom funkcija sa eksplicitnim pomerajem. (Deo c — upis u novu datoteku po šemi sa slike — nije obuhvaćen.)

main.c

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>
#define N 105
#define MASTER 0

int main(int argc, char** argv)
{
    int rank, size;
    MPI_File fh;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* redosled u fajlu: od POSLEDNJEG ka PRVOM procesu -> offset unazad */
    MPI_Offset offset = (MPI_Offset)(size - 1 - rank) * N * sizeof(int);

    int* buf = (int*)malloc(N * sizeof(int));
    for (int i = 0; i < N; i++)
        buf[i] = rank; /* proizvoljni celi brojevi */

    /* a) upis POJEDINACNIM POKAZIVACEM: MPI_File_seek + MPI_File_write */
    MPI_File_open(MPI_COMM_WORLD, "file1.dat",
        MPI_MODE_CREATE | MPI_MODE_WRONLY, MPI_INFO_NULL, &fh);
    MPI_File_seek(fh, offset, MPI_SEEK_SET);
    MPI_File_write(fh, buf, N, MPI_INT, MPI_STATUS_IGNORE);
    MPI_File_close(&fh);
    MPI_Barrier(MPI_COMM_WORLD);

    /* b) ponovo otvoriti; citanje EKSPPLICITNIM POMERAJEM: MPI_File_read_at
       (NE ide seek uz _at funkcije!) */
    int* readBuf = (int*)malloc(N * sizeof(int));
    MPI_File_open(MPI_COMM_WORLD, "file1.dat",
        MPI_MODE_RDONLY, MPI_INFO_NULL, &fh);
    MPI_File_read_at(fh, offset, readBuf, N, MPI_INT, MPI_STATUS_IGNORE);
    MPI_File_close(&fh);

    free(buf);
    free(readBuf);
    MPI_Finalize();
    return 0;
}
```


MPI — OKT2 2025 — Čitanje 10MB datoteke — pojedinačni pokazivači, osmobitni tip (prvi deo)

Zadatak: Datoteka input.dat sadrži ukupno 10MB podataka. Napisati MPI program koji vrši obradu ovih podataka i priprema ih za obradu, ujedno vršeći paralelni upis i čitanje datoteke. Na početku, svi procesi vrše čitanje iste količine podataka tako da poslednji proces čita prvi skup podataka, preposlednji proces drugi skup, itd. korišćenjem funkcija sa pojedinačnim pokazivačem. Napomena: Zadatak rešiti korišćenjem osmobitnog tipa podataka. (Deo sa upisom u output.dat po šemi sa slike nije obuhvaćen.)

main.c

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>
#define FILESIZE (10*1024*1024)    /* 10MB */
#define MASTER 0

int main(int argc, char** argv)
{
    int rank, size;
    MPI_File fh;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* osmobitni tip podataka -> char; svi citaju istu kolicinu */
    int bufsize = FILESIZE / size;          /* bajtova po procesu */
    char* buf = (char*)malloc(bufsize);

    /* poslednji proces cita PRVI skup, preposlednji DRUGI, itd.
       -> offset unazad */
    MPI_Offset offset = (MPI_Offset)(size - 1 - rank) * bufsize;

    /* citanje POJEDINACNIM POKAZIVACEM: MPI_File_seek + MPI_File_read */
    MPI_File_open(MPI_COMM_WORLD, "input.dat",
        MPI_MODE_RDONLY, MPI_INFO_NULL, &fh);
    MPI_File_seek(fh, offset, MPI_SEEK_SET);
    MPI_File_read(fh, buf, bufsize, MPI_CHAR, MPI_STATUS_IGNORE);
    MPI_File_close(&fh);
    MPI_Barrier(MPI_COMM_WORLD);

    free(buf);
    MPI_Finalize();
    return 0;
}
```

MPI — JUN 2026 — Čitanje 1kB datoteke — eksplicitni pomeraji (prvi deo; isti obrazac: JUL 2026)

Zadatak: Datoteka input.dat sadrži ukupno 1kB podataka. Napisati MPI program koji vrši obradu ovih podataka i priprema ih za vizualizaciju, ujedno vršeći paralelni upis i čitanje datoteke. Na početku, svi procesi vrše čitanje iste količine podataka tako da prvi proces čita poslednji skup podataka, drugi proces pretposlednji skup, itd. korišćenjem funkcija sa eksplicitnim pomerajem. (Deo sa upisom u output.dat po šemi sa slike nije obuhvaćen.)

main.c

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>
#define FILESIZE 1024 /* 1kB */
#define MASTER 0

int main(int argc, char** argv)
{
    int rank, size;
    MPI_File fh;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* svi citaju istu kolicinu podataka */
    int bufsz = FILESIZE / size;
    char* buf = (char*)malloc(bufsz);

    /* prvi proces cita POSLEDNJI skup, drugi PRETPOSLEDNJI, itd.
       -> offset unazad */
    MPI_Offset offset = (MPI_Offset)(size - 1 - rank) * bufsz;

    /* citanje EKSPPLICITNIM POMERAJEM: MPI_File_read_at (NE ide seek) */
    MPI_File_open(MPI_COMM_WORLD, "input.dat",
        MPI_MODE_RDONLY, MPI_INFO_NULL, &fh);
    MPI_File_read_at(fh, offset, buf, bufsz, MPI_CHAR, MPI_STATUS_IGNORE);
    MPI_File_close(&fh);
    MPI_Barrier(MPI_COMM_WORLD);

    free(buf);
    MPI_Finalize();
    return 0;
}
```


WCF

WCF — JUN 2024 — Sistem za zakup skladišta

Zadatak: Koristeći WCF kreirati sistem za zakup skladišta. Potrebno je da servis podržava sledeće funkcionalnosti: zakup skladišta (proslediti Vlasnika(ime, prezime i jmbg) i skladište (IdSkladišta, početak zakupa, kraj zakupa, cena)); vraća listu svih aktivnih skladišta zadatog vlasnika; vraća listu svih vlasnika aktivnih skladišta; vraća listu svih skladišta i istoriju njihovih vlasnika (zakupa). Obavezno izdvojiti interfejs, implementaciju, web.config (dovoljan je samo deo za setovanje servisa) i klijentsku stranu koja demonstrira rad servisa.

IZakupSkladista.cs

```
[ServiceContract]
public interface IZakupSkladista {
    [OperationContract]
    void ZakupiSkladiste(Vlasnik vlasnik, Skladiste skladiste);

    [OperationContract]
    List<Skladiste> VratiListuAktivnihSkladista(string jmbg);

    [OperationContract]
    List<Vlasnik> VratiListuVlasnikaAktivnihSkladista();

    [OperationContract]
    List<Zakup> VratiIstorijuZakupa();
}
```

Vlasnik.cs

```
[DataContract]
public class Vlasnik {
    [DataMember]
    public string Ime { get; set; }

    [DataMember]
    public string Prezime { get; set; }

    [DataMember]
    public string JMBG { get; set; }
}
```

Skladiste.cs

```
[DataContract]
public class Skladiste {
    [DataMember]
    public int IdSkladista { get; set; }

    [DataMember]
    public DateTime PocetakZakupa { get; set; }

    [DataMember]
    public DateTime KrajZakupa { get; set; }

    [DataMember]
    public decimal Cena { get; set; }
}
```

Zakup.cs

```
[DataContract]
public class Zakup {
```

```

[DataMember]
public Vlasnik Vlasnik { get; set; }

[DataMember]
public Skladiste Skladiste { get; set; }
}

```

ZakupSkladista.svc.cs

```

[ServiceBehavior(InstanceContextMode = InstanceContextMode.Single)]
public class ZakupSkladista : IZakupSkladista {
    private static List<Zakup> zakupi = new List<Zakup>();

    public void ZakupiSkladiste(Vlasnik vlasnik, Skladiste skladiste) {
        zakupi.Add(new Zakup{
            Vlasnik = vlasnik,
            Skladiste = skladiste
        });
    }

    public List<Skladiste> VratiListuAktivnihSkladista(string jmbg) {
        return zakupi.Where(x => x.Skladiste.PocetakZakupa <= DateTime.Now &&
            x.Skladiste.KrajZakupa >= DateTime.Now &&
            x.Vlasnik.JMBG == jmbg)
            .Select(x => x.Skladiste)
            .ToList();
    }

    public List<Vlasnik> VratiListuVlasnikaAktivnihSkladista() {
        return zakupi.Where(x => x.Skladiste.PocetakZakupa <= DateTime.Now &&
            x.Skladiste.KrajZakupa >= DateTime.Now)
            .Select(x => x.Vlasnik)
            .Distinct()
            .ToList();
    }

    public List<Zakup> VratiIstorijuZakupa() {
        return zakupi.ToList();
    }
}

```

web.config

```

<configuration>
  <system.serviceModel>
    <services>
      <service name="Jun24.ZakupSkladista">
        <endpoint contract="Jun24.IZakupSkladista" binding="basicHttpBinding" />
      </service>
    </services>
  </system.serviceModel>
</configuration>

```

Client.cs

```

public class Client {
    public static void Main(string[] args) {
        ZakupSkladistaClient proxy = new ZakupSkladistaClient();

        Vlasnik petar = new Vlasnik {
            Ime = "Petar",
            Prezime = "Petrovic",
            JMBG = "123543"
        };

        Skladiste skladiste = new Skladiste {
            IdSkladista = 1,

```

```

        PocetakZakupa = DateTime.Now,
        KrajZakupa = DateTime.Now.AddDays(30),
        Cena = 213.0m
    };

    proxy.ZakupiSkladiste(petar, skladiste);

    Console.WriteLine("Aktivna skladista vlasnika:");
    foreach (Skladiste s in proxy.VratiListuAktivnihSkladista("123543"))
        Console.WriteLine($"Id: {s.IdSkladista}, cena: {s.Cena}, od {s.PocetakZakupa} do {s.KrajZakupa}");

    Console.WriteLine("Vlasnici aktivnih skladista:");
    foreach (Vlasnik v in proxy.VratiListuVlasnikaAktivnihSkladista())
        Console.WriteLine($"{v.Ime} {v.Prezime}, jmbg: {v.JMBG}");

    Console.WriteLine("Istorija zakupa:");
    foreach (Zakup z in proxy.VratiIstorijuZakupa())
        Console.WriteLine($"Skladiste {z.Skladiste.IdSkladista} - {z.Vlasnik.Ime} {z.Vlasnik.Prezime}");

    proxy.Close();
}

```

WCF — OKT2 2025 — Full-duplex kalkulator sa callback-om (isti zadatak: OKT 2025 / APR 2025)

Zadatak: Koristeći WCF kreirajte full-duplex sistem kalkulatora. Korisnik u svojoj sesiji može da obriše trenutno računanje, doda broj, oduzme broj, pomnoži brojem i podeli rezultat prosleđenim brojem. Svaka operacija se odmah izvršava nad rezultatom (prethodni rezultat) i smešta u rezultat. Operacija vraća rezultat izvršene. Servis po izvršenju operacije poziva klijenta i prosleđuje mu do tog momenta kreirani izraz (Na primer: izraz 2+3-5*7). Obavezno izdvojiti interfejs, implementaciju, web.config (dovoljan je samo deo za setovanje servisa i na klijentskoj strani deo za callback) i klijentsku stranu koja demonstrira rad servisa.

IKalkulator.cs

```
[ServiceContract(CallbackContract = typeof(IKalkulatorCallback), SessionMode = SessionMode.Required)]
public interface IKalkulator {
    [OperationContract]
    double ObrisiRacunicu();

    [OperationContract]
    double DodajBroj(int broj);

    [OperationContract]
    double OduzmiBroj(int broj);

    [OperationContract]
    double PomnoziBrojem(int broj);

    [OperationContract]
    double PodeliBroj(int broj);
}
```

IKalkulatorCallback.cs

```
public interface IKalkulatorCallback {
    [OperationContract(IsOneWay = true)]
    void PrikaziIzraz(string izraz);
}
```

Kalkulator.svc.cs

```
[ServiceBehavior(InstanceContextMode = InstanceContextMode.PerSession)]
public class Kalkulator : IKalkulator {
    private string koraci = "0";
    private double rezultat = 0;

    private IKalkulatorCallback callback;

    public Kalkulator() {
        callback = OperationContext.Current.GetCurrentCallbackChannel<IKalkulatorCallback>();
    }

    public double ObrisiRacunicu() {
        koraci = "0";
        rezultat = 0;

        callback.PrikaziIzraz(koraci);

        return rezultat;
    }

    public double DodajBroj(int broj) {
```

```

    rezultat += broj;
    koraci += "+" + broj;

    callback.PrikaziIzraz(koraci);

    return rezultat;
}

public double OduzmiBroj(int broj) {
    rezultat -= broj;
    koraci += "-" + broj;

    callback.PrikaziIzraz(koraci);

    return rezultat;
}

public double PomnoziBrojem(int broj) {
    rezultat *= broj;
    koraci += "*" + broj;

    callback.PrikaziIzraz(koraci);

    return rezultat;
}

public double PodeliBroj(int broj) {
    if (broj == 0) return rezultat;

    rezultat /= broj;
    koraci += "/" + broj;

    callback.PrikaziIzraz(koraci);

    return rezultat;
}
}

```

web.config

```

<configuration>
  <system.serviceModel>
    <services>
      <service name="Okt225.Kalkulator">
        <endpoint contract="Okt225.IKalkulator" binding="wsDualHttpBinding" />
      </service>
    </services>
    <bindings>
      <wsDualHttpBinding>
        <binding name="wsDualHttpBindingConfiguration" transactionFlow="true" />
      </wsDualHttpBinding>
    </bindings>
    <protocolMapping>
      <add scheme="http" binding="wsDualHttpBinding" bindingConfiguration="wsDualHttpBin
        dingConfiguration"/>
    </protocolMapping>
  </system.serviceModel>
</configuration>

```

WCF — JUN 2026 — Kalkulator sa prosleđivanjem izuzetka (FaultContract)

Zadatak: Koristeći WCF kreirajte sistem kalkulatora. Korisnik u svojoj sesiji može da obriše trenutno računanje, doda broj, oduzme broj, pomnoži brojem i podeli rezultat prosleđenim brojem. Svaka operacija se odmah izvršava nad rezultatom (prethodni rezultat) i smešta u rezultat. Operacija vraća rezultat izvršenja i do tog momenta kreirani izraz (Na primer: izraz $2+3-5*7$). Predvideti prosleđivanje izuzetaka klijentu. Obavezno izdvojiti interfejs, implementaciju, web.config (dovoljan je samo deo za setovanje) i klijentsku stranu koja demonstrira rad servisa. Klijentska strana mora pozvati sve metode servisa i prikazati njihov rezultat ako postoji.

IKalkulator.cs

```
[ServiceContract]
public interface IKalkulator {
    [OperationContract]
    Response ObrisiRacunicu();

    [OperationContract]
    Response DodajBroj(int broj);

    [OperationContract]
    Response OduzmiBroj(int broj);

    [OperationContract]
    Response PomnoziBrojem(int broj);

    [OperationContract]
    [FaultContract(typeof(KalkulatorGreska))]
    Response PodeliRezultatBrojem(int broj);
}
```

Response.cs

```
[DataContract]
public class Response {
    [DataMember]
    public int Rezultat { get; set; }

    [DataMember]
    public string Koraci { get; set; }
}
```

KalkulatorGreska.cs

```
[DataContract]
public class KalkulatorGreska {
    [DataMember]
    public string Poruka { get; set; }
}
```

Kalkulator.svc.cs

```
[ServiceBehavior(InstanceContextMode = InstanceContextMode.PerSession)]
public class Kalkulator : IKalkulator {
    private Response response = new Response { Rezultat = 0, Koraci = "" };

    public Response ObrisiRacunicu() {
        response.Rezultat = 0;
        response.Koraci = "";

        return response;
    }
}
```

```

public Response DodajBroj(int broj) {
    response.Rezultat += broj;
    response.Koraci += "+" + broj;

    return response;
}

public Response OduzmiBroj(int broj) {
    response.Rezultat -= broj;
    response.Koraci += "-" + broj;

    return response;
}

public Response PomnoziBrojem(int broj) {
    response.Rezultat *= broj;
    response.Koraci += "*" + broj;

    return response;
}

public Response PodeliRezultatBrojem(int broj) {
    if(broj == 0) {
        throw new FaultException<KalkulatorGreska>(
            new KalkulatorGreska { Poruka = "Deljenje nulom nije dozvoljeno " }
        );
    }

    response.Rezultat /= broj;
    response.Koraci += "/" + broj;

    return response;
}
}

```

web.config

```

<configuration>
  <system.serviceModel>
    <services>
      <service name="Jun26.Kalkulator">
        <endpoint contract="Jun26.IKalkulator" binding="basicHttpBinding" />
      </service>
    </services>
  </system.serviceModel>
</configuration>

```

Client.cs

```

public class Client {
    public static void Main(string[] args) {
        KalkulatorClient proxy = new KalkulatorClient();

        var rez1 = proxy.DodajBroj(3);
        Console.WriteLine($"Rezultat je {rez1.Rezultat} , Koraci : {rez1.Koraci}");

        var rez2 = proxy.PomnoziBrojem(9);
        Console.WriteLine($"Rezultat je {rez2.Rezultat} , Koraci : {rez2.Koraci}");

        var rez3 = proxy.OduzmiBroj(5);
        Console.WriteLine($"Rezultat je {rez3.Rezultat} , Koraci : {rez3.Koraci}");

        try {
            proxy.PodeliRezultatBrojem(0);
        }
        catch (FaultException<KalkulatorGreska> ex) {
            Console.WriteLine($"Greska: {ex.Detail.Poruka}");
        }
    }
}

```

```
}

var rez4 = proxy.PodeliRezultatBrojem(3);
Console.WriteLine($"Rezultat je {rez4.Rezultat} , Koraci : {rez4.Koraci}");

var rez5 = proxy.ObrisiRacunicu();
Console.WriteLine($"Rezultat je {rez5.Rezultat} , Koraci : {rez5.Koraci}");

proxy.Close();
}
}
```